

INGETIS

Ecole d'ingenierie informatique — 62 bis rue Gay-Lussac, 75005 Paris
EADL - RNCP38822

DOSSIER PROFESSIONNEL

Soutenance de certification RNCP Niveau 7

BLOC 2 — CONCEVOIR ET DEVELOPPER DES SOLUTIONS

Specialite : Developpement Logiciel

Candidat(e) : _____	Entreprise : _____
Numero apprenant : _____	Maitre d'apprentissage : _____
Annee : 2025-2026	Tuteur pedagogique : _____

Date de soutenance : _____ / _____ / _____	Score obtenu : _____ / 20
--	----------------------------------

*Soutenance : 20 min de presentation + 20 min d'entretien avec le jury
Document confidentiel — INGETIS 2025-2026 — Usage academique strictement reserve*

Compétences transversales — Ces deux compétences doivent être démontrées dans chaque bloc et à chaque soutenance.

(1) Maîtrise de l'anglais technique : veille technologique à partir de sources anglophones, références anglophones intégrées au dossier; capacité d'échange en anglais si sollicité par le jury.

(2) Numérique responsable : intégration des principes d'eco-conception, analyse de l'impact environnemental des solutions proposées, conformité aux réglementations en vigueur.

Introduction. Présentation du Bloc 2

Le présent dossier professionnel rend compte de la conception et du développement d'ECOTRACK, une plateforme logicielle de gestion intelligente des déchets urbains, réalisée dans le cadre du projet fil rouge de l'année de Mastère 1. Il retrace l'intégralité du cycle de développement de la solution, depuis la définition de l'architecture logicielle jusqu'à la mise en œuvre des tests automatisés et à la production de la documentation technique.

Le contexte professionnel est celui d'une collectivité de 500 000 habitants confrontée à une gestion des déchets coûteuse et peu réactive : les tournées de collecte sont planifiées à fréquence fixe, sans connaissance du taux de remplissage réel des 2 000 conteneurs du territoire. Il en résulte des collectes inutiles, des débordements et un impact environnemental évitable. La problématique centrale adressée par ce projet peut être formulée ainsi : comment concevoir une application capable d'exploiter en temps réel les données de 2 000 capteurs ultrasoniques (une mesure toutes les 15 minutes, soit environ 500 000 messages par jour) pour piloter la collecte par la donnée plutôt que par le calendrier, tout en engageant les citoyens dans une démarche participative ?

Pour y répondre, ce dossier s'articule en sept chapitres. Le chapitre 2 cadre le projet (organisation, objectifs fonctionnels, choix de la stack). Le chapitre 3 détaille l'architecture logicielle et le modèle de données. Les chapitres 4 et 5 documentent respectivement le développement front-end et back-end, ainsi que la sécurisation de l'application. Le chapitre 6 présente la stratégie de tests et d'assurance qualité, le chapitre 7 inventorie les livrables documentaires, et le chapitre 8 dresse le bilan du projet et ses perspectives d'évolution. Les deux compétences transversales — maîtrise de l'anglais technique et numérique responsable — sont intégrées de manière continue dans l'ensemble de ces chapitres.

2. Contexte et cadrage du projet

2.1 Présentation de l'organisation et de la problématique

Le projet s'inscrit dans une mission simulant un contexte professionnel réel : la direction de la gestion des déchets d'une métropole souhaite moderniser son service de collecte. L'organisation gère un parc de 2 000 conteneurs répartis sur l'ensemble du territoire urbain, collectés par une flotte de véhicules selon des tournées historiques rarement remises en question.

La problématique métier est double. D'une part, l'absence de visibilité sur le niveau de remplissage conduit à collecter des conteneurs quasi vides tout en laissant déborder les points les plus sollicités, générant nuisances et plaintes citoyennes. D'autre part, les tournées fixes engendrent des kilomètres inutiles, donc des coûts de carburant et de main-d'œuvre élevés et des émissions de CO₂ superflues. ECOTRACK vise à transformer cette logique calendaire en une logique pilotée par la donnée temps réel, en fournissant aux gestionnaires une cartographie vivante des niveaux, aux agents de collecte des feuilles de route optimisées, et aux 15 000 citoyens un canal de signalement gamifié. La plateforme sert quatre profils (citoyen, agent, gestionnaire, administrateur) à travers trois applications : un tableau de bord web et deux applications mobiles (citoyen et agent).

2.2 Objectifs fonctionnels et perimetre

Le périmètre fonctionnel d'ECOTRACK s'organise autour de quatre familles de fonctionnalités, priorisées selon la méthode MoSCoW.

Fonctionnalités essentielles (MUST) : ingestion en temps réel des mesures des capteurs via MQTT ; cartographie temps réel des conteneurs et de leur niveau de remplissage ; génération d'alertes automatiques au franchissement de seuils ; calcul de tournées optimisées par algorithme du voyageur de commerce (TSP + 2-opt) ; signalement citoyen d'un conteneur plein (avec photo) depuis l'application mobile ; module de gamification (points, badges, défis, classement) ; authentification sécurisée avec double facteur (MFA) et contrôle d'accès basé sur quatre rôles (Citoyen, Agent, Gestionnaire, Administrateur).

Fonctionnalités importantes (SHOULD) : tableau de bord analytique (historique, tendances de remplissage) ; application mobile agent (réception de tournée, scan QR, géolocalisation) ; notifications multicanal (e-mail, push) ; export de rapports. Fonctionnalités optionnelles (COULD) : mode hors-ligne de l'application agent (PWA / Service Workers).

Les contraintes non fonctionnelles structurantes sont : un temps de réponse des API inférieur à 500 ms au 95e centile (et des requêtes SQL sous 200 ms pour 90 % d'entre elles), la prise en charge de 500 utilisateurs simultanés, une disponibilité cible de 99,5 %, le calcul d'une tournée de 50 conteneurs en quelques secondes, une couverture de tests supérieure à 60 %, une conformité RGPD des données de géolocalisation et le respect du référentiel d'accessibilité WCAG 2.1 niveau AA. À titre d'exemple, un critère d'acceptation de la user story « consulter la carte temps réel » impose que toute mesure remontée soit reflétée sur la carte en moins de deux secondes.

2.3 Choix de la stack technologique

Les choix technologiques ont été arrêtés à l'issue d'une veille comparative (cf. dossier de veille du Bloc 1), sur la base de cinq critères : maturité de l'écosystème, adéquation au besoin, performances, compétences disponibles et considérations d'éco-conception.

Côté front-end, React 18 associé à TypeScript a été retenu face à Vue.js et Angular pour le tableau de bord web, et React Native pour les deux applications mobiles (citoyen et agent), afin de mutualiser les compétences et le Design System ; s'ajoutent React-Leaflet (cartographie sur fonds OpenStreetMap) et Recharts (visualisation). Côté back-end, une approche hybride a été privilégiée : FastAPI (Python) pour l'API métier et le service d'optimisation, tirant parti de l'écosystème scientifique Python pour l'algorithme TSP, et Node.js pour les services temps réel (ingestion IoT, notifications), performant sur les charges d'entrées/sorties intensives.

Le stockage repose sur PostgreSQL 15 étendu par PostGIS pour les requêtes géospatiales et les séries temporelles de mesures (partitionnement temporel). Redis assure la mise en cache, les sessions et les classements de gamification, tandis qu'un bus événementiel Apache Kafka découple l'ingestion IoT et la communication asynchrone entre services. L'ensemble est containerisé avec Docker, exposé derrière une API Gateway, et cible une orchestration Kubernetes en M2. Le choix de briques open source et la mutualisation des conteneurs participent directement à la démarche de numérique responsable, en limitant l'empreinte d'infrastructure.

3. Architecture logicielle

3.1 Modèle d'architecture global

ECOTRACK adopte une architecture microservices orientée Domain-Driven Design (DDD), afin de garantir modularité, scalabilité horizontale et résilience. Le diagramme d'architecture global (C4 niveau 2) est présenté en Annexe A et référencé ci-après.

La couche de présentation regroupe trois applications clientes : un tableau de bord web React (gestionnaires et administrateurs) et deux applications mobiles React Native (citoyen et agent), qui communiquent avec le back-end via une API Gateway et reçoivent les mises à jour temps réel par WebSocket. La couche métier est découpée en microservices correspondant aux Bounded Contexts du domaine : Ingestion IoT (réception/validation des trames MQTT), Optimisation des tournées (TSP + 2-opt), Authentification (JWT, MFA, RBAC), Signalement citoyen, Gamification (points, badges, défis, classement) et Notifications. Ces services communiquent de façon asynchrone via le bus Apache Kafka. La couche de données s'appuie sur PostgreSQL/PostGIS (avec partitionnement temporel des mesures) et sur Redis pour le cache.

Les services sont containerisés via Docker et exposés derrière une API Gateway assurant terminaison TLS, authentification, contrôle RBAC et rate limiting (l'orchestration Kubernetes étant la cible de la phase M2). Ce découpage permet de faire évoluer ou monter en charge indépendamment les composants les plus sollicités — typiquement l'ingestion IoT ou l'optimisation — sans impacter le reste de la plateforme.

3.2 Patterns de conception appliques

Plusieurs patterns architecturaux et de conception structurent le code afin d'en garantir la lisibilité, la testabilité et l'évolutivité.

Le pattern Repository abstrait l'accès aux données : la logique métier manipule des interfaces de persistance sans dépendre du SGBD, ce qui facilite les tests unitaires par injection de doubles. Le pattern Strategy est appliqué au moteur d'optimisation : l'heuristique gloutonne du MVP et l'algorithme génétique envisagé en évolution sont interchangeables derrière une même interface, sans modification du code appelant.

Le pattern Observer sous-tend le mécanisme de publication/abonnement des WebSockets : lorsqu'une mesure franchit un seuil, les clients abonnés à la zone concernée sont notifiés en temps réel. Le pattern Factory centralise la création des objets métier (alertes, tournées) selon leur type. Enfin, l'organisation générale s'inspire des principes de la Clean Architecture, en isolant le domaine métier des détails techniques (frameworks, base de données, transport), ce qui limite l'adhérence technologique et la dette à long terme.

3.3 Modele de donnees

Le modèle de données relationnel s'organise autour des entités principales : Utilisateur (quatre rôles), Conteneur, Zone, Mesure, Tournée, Signalement, Alerte, ainsi que les entités de gamification (Point, Badge, Défi, Classement). Le diagramme entité-relation complet est fourni en Annexe B.

La table containers stocke les caractéristiques et la position géographique de chaque conteneur, à l'aide d'une colonne géométrique PostGIS (type POINT, SRID 4326) indexée par un index spatial GiST pour accélérer les requêtes de proximité utilisées par l'optimisation des tournées ; la table zones associe chaque conteneur à l'un des 12 secteurs. La table measurements, qui concentre l'essentiel du volume (~500 000 lignes par jour), est partitionnée par mois (PARTITION BY RANGE sur l'horodatage), ce qui maintient des performances stables à fort volume et facilite l'archivage par tranche.

Des stratégies d'optimisation complémentaires sont mises en œuvre : agrégats continus pré-calculés (moyennes horaires et journalières) pour alimenter les tableaux de bord sans rejouer l'historique, compression automatique des données anciennes pour réduire l'empreinte de stockage, et politiques de rétention différenciées entre données brutes et agrégats. Ces choix illustrent une démarche d'éco-conception au niveau de la persistance.

4. Développement front-end

4.1 Design System et composants

L'interface repose sur un Design System cohérent garantissant l'homogénéité visuelle et la réutilisabilité des composants. Les maquettes Figma de référence sont jointes en Annexe C.

La palette chromatique associe une couleur primaire sobre à un code couleur fonctionnel pour les niveaux de remplissage (vert : faible, orange : élevé, rouge : saturé), choisi pour rester distinguable y compris en cas de déficience de perception des couleurs. La typographie privilégie une police sans empattement de haute lisibilité. La bibliothèque de composants combine Material UI (MUI) et des utilitaires Tailwind CSS : boutons, cartes d'information, badges d'état, modales et composants cartographiques sont factorisés et documentés, ce qui accélère le développement et fiabilise la cohérence d'ensemble.

4.2 Accessibilité et conformité WCAG

L'accessibilité numérique constitue une exigence à la fois réglementaire et éthique. Le développement a pris pour référence le niveau AA du référentiel WCAG 2.1.

Les mesures implémentées incluent : le respect des ratios de contraste minimaux (4,5:1 pour le texte courant), une navigation entièrement possible au clavier avec un ordre de tabulation logique et des états de focus visibles, l'emploi d'attributs ARIA pour décrire les composants dynamiques (carte, alertes, menus), des alternatives textuelles pour les éléments non textuels et une structuration sémantique correcte des titres. La cartographie, intrinsèquement visuelle, est doublée d'une vue liste accessible présentant les mêmes informations. Des tests ont été menés avec un lecteur d'écran (NVDA) et l'auditeur automatique axe-core afin de valider la conformité et de corriger les écarts résiduels.

4.3 Performance et optimisation

Les performances perçues de l'interface ont été mesurées avec Lighthouse et suivies via les Core Web Vitals (LCP, FID/INP, CLS), avec pour objectif un score de performance supérieur à 90 sur les parcours critiques.

Plusieurs optimisations ont été appliquées : découpage du code (code splitting) et chargement différé (lazy loading) des vues et des composants lourds comme la carte ; mise en cache des réponses d'API côté client et via Redis côté serveur ; optimisation et compression des ressources statiques ; et limitation du volume de données transmises grâce aux agrégats pré-calculés. Le rafraîchissement temps réel via WebSocket évite par ailleurs le coûteux polling périodique. Outre le gain d'expérience utilisateur, ces optimisations réduisent la consommation réseau et énergétique, conformément aux principes d'éco-conception.

5. Développement back-end et conception de l'API

5.1 Architecture du serveur et des services

Chaque microservice back-end suit une organisation modulaire en couches : une couche de routage (endpoints), une couche de services (logique métier), une couche d'accès aux données (repositories) et une couche de modèles. Cette séparation des responsabilités facilite la maintenance et les tests.

Le service d'API métier, développé avec FastAPI, exploite les modèles Pydantic pour la validation automatique des entrées et la sérialisation des sorties, ainsi que l'injection de dépendances native du framework. La persistance s'appuie sur l'ORM SQLAlchemy. Une chaîne de middlewares prend en charge les préoccupations transverses : journalisation structurée, gestion centralisée des erreurs, CORS, limitation du débit (rate limiting à 100 requêtes par minute et par IP) et vérification des jetons d'authentification. Les services temps réel en Node.js consomment les trames MQTT et diffusent les événements via Socket.io.

5.2 Conception et documentation de l'API

L'API suit le style REST et respecte des conventions de nommage explicites : ressources au pluriel (/containers, /routes, /alerts), verbes HTTP porteurs de sémantique (GET, POST, PUT, PATCH, DELETE) et filtrage par paramètres de requête. Le versionnage est assuré par préfixe d'URL (/api/v1). La documentation est générée automatiquement au format OpenAPI/Swagger et fournie en Annexe D.

Quelques endpoints significatifs illustrent la conception : GET /api/v1/containers retourne l'état des conteneurs avec filtres géographiques ; POST /api/v1/routes/optimize déclenche le calcul d'une tournée optimisée ; GET /api/v1/alerts liste les alertes actives. La gestion des erreurs est normalisée : codes de retour HTTP cohérents (200, 201, 400, 401, 403, 404, 422, 429, 500) et corps d'erreur structuré comportant un code applicatif, un message et, le cas échéant, le détail des champs invalides. Cette rigueur facilite l'intégration côté front-end et la supervision.

5.3 Sécurisation de l'application (OWASP)

La sécurisation de l'application a été conduite en regard de l'OWASP Top 10, dès la conception (security by design).

L'authentification repose sur des jetons JWT (jeton d'accès de courte durée de 15 minutes et jeton de rafraîchissement de 7 jours) complétés par une double authentification TOTP. L'autorisation est gérée par un contrôle d'accès basé sur les rôles (RBAC : Citoyen, Agent, Gestionnaire, Administrateur), appliqué côté serveur pour prévenir les défaillances de contrôle d'accès. La protection contre les injections est assurée par le recours systématique à des requêtes paramétrées via l'ORM et à la validation stricte des entrées par Pydantic.

Les autres mesures incluent : une politique CORS restrictive, l'ajout d'en-têtes de sécurité HTTP (Content-Security-Policy, HSTS, X-Content-Type-Options), la gestion des secrets hors du code source (variables d'environnement et coffre-fort de secrets), le hachage des mots de passe avec un algorithme à facteur de coût (bcrypt), le chiffrement des échanges via TLS et la limitation de débit contre les attaques par force brute et par déni de service applicatif. Les données personnelles, notamment de géolocalisation, font l'objet d'une minimisation et d'un chiffrement conformément au RGPD.

6. Strategie de tests et assurance qualite

6.1 Pyramide de tests et objectifs

La stratégie de tests adopte une approche pyramidale, privilégiant un socle large de tests unitaires rapides, complété par des tests d'intégration et, au sommet, un nombre restreint de tests de bout en bout (end-to-end) sur les parcours critiques. Les rapports de couverture sont fournis en Annexe E.

Les tests unitaires (Pytest côté Python, Jest côté Node et React) ciblent la logique métier — notamment l'algorithme d'optimisation et les règles de déclenchement d'alertes — avec un objectif de couverture supérieur à 60 % (seuil minimal fixé par le cahier des charges). Les tests d'intégration valident les interactions entre services et avec la base de données, exécutés sur des conteneurs éphémères. Les tests end-to-end (Playwright) rejouent les scénarios utilisateurs majeurs : authentification 2FA, consultation de la carte, génération d'une tournée. Une approche orientée tests (TDD) a été appliquée sur les composants métier sensibles. L'ensemble de la pyramide est rejoué automatiquement par la chaîne d'intégration continue à chaque modification.

6.2 Tests de securite

La sécurité fait l'objet de tests automatisés intégrés à la chaîne CI/CD. L'analyse statique du code (SAST) est réalisée avec Bandit (Python) et l'analyse de qualité SonarQube, qui détectent les motifs de code dangereux et les vulnérabilités potentielles avant exécution.

L'analyse dynamique (DAST) est effectuée à l'aide d'OWASP ZAP contre l'environnement de pré-production, afin d'éprouver l'application en fonctionnement (injections, en-têtes, configuration). L'audit des dépendances est assuré en continu par npm audit, pip-audit et Dependabot, complétés par le scan des images Docker avec Trivy. Les vulnérabilités identifiées sont qualifiées par criticité puis corrigées de manière priorisée ; les correctifs de dépendances sont automatisés autant que possible pour réduire le délai d'exposition.

7. Documentation technique et livrables

La documentation technique constitue un livrable à part entière du projet et conditionne sa maintenabilité.

Les livrables documentaires produits comprennent : un fichier README complet (présentation, prérequis, démarrage rapide) ; un guide d'installation et de déploiement détaillant la procédure de containerisation ; un guide du contributeur précisant les conventions de code, la stratégie de branches et le processus de revue ; la documentation de l'API générée au format OpenAPI/Swagger et tenue à jour automatiquement ; un guide utilisateur destiné aux gestionnaires ; et un ensemble d'Architecture Decision Records (ADR) traçant les principales décisions de conception et leurs justifications. La documentation s'appuie largement sur des sources et références anglophones, en cohérence avec la compétence transversale d'anglais technique. À la date de rédaction, l'ensemble de ces livrables est complet, à l'exception du guide utilisateur en cours de finalisation.

8. Conclusion et perspectives

Le projet ECOTRACK a permis de concevoir et de développer une solution applicative complète répondant à la problématique posée : faire passer la gestion des déchets d'une logique calendaire à un pilotage par la donnée temps réel. Les objectifs fonctionnels essentiels ont été atteints — ingestion IoT, cartographie temps réel, alertes et optimisation des tournées — au sein d'une architecture microservices scalable, sécurisée selon l'OWASP Top 10 et couverte par une stratégie de tests automatisés.

Sur le plan des difficultés, la maîtrise de la montée en charge de l'ingestion et l'optimisation des requêtes sur les séries temporelles ont constitué les principaux défis, résolus par le recours à un partitionnement temporel PostgreSQL, à un bus Kafka et à une mise en cache appropriée. Le calibrage de l'algorithme TSP a imposé une démarche itérative, justifiant le choix d'une heuristique gloutonne pour le MVP.

Ce projet a été l'occasion de développer des compétences transversales d'architecture, de sécurité applicative et de qualité logicielle, ainsi que de pratiquer l'anglais technique à travers la veille et la documentation. Plusieurs perspectives se dessinent : l'introduction d'un modèle prédictif de remplissage pour anticiper les collectes, le passage de l'heuristique gloutonne à un algorithme génétique, le déploiement d'une application mobile chauffeur en mode hors-ligne, et l'extension de la plateforme à d'autres collectivités dans une logique de mutualisation et d'open data.

Modalites. Volume attendu et instructions de remise

Le present dossier professionnel doit etre transmis a l'equipe pedagogique dans un delai de deux a quatre semaines avant la date de soutenance, selon le calendrier communique par votre responsable de formation. Tout dossier remis hors delai pourra entrainer le report de la soutenance.

Volume minimal attendu : 25 pages hors annexes (maximum conseille : 40 pages)

Chapitre	Min.	Max.
Introduction — Contexte et problematique	1p	3p
Chapitre 2 — Contexte et cadrage du projet	3p	6p
Chapitre 3 — Architecture logicielle	5p	7p
Chapitre 4 — Developpement front-end	3p	5p
Chapitre 5 — Back-end et conception de l'API	5p	7p
Chapitre 6 — Strategie de tests et qualite	3p	5p
Chapitre 7 — Documentation technique	2p	3p
Chapitre 8 — Conclusion et perspectives	1p	2p
TOTAL hors annexes : 25 pages minimum — 40 pages maximum conseille		

Le document doit etre soumis en format PDF natif, nomme selon la convention suivante :

NOM_Prenom_BLOC [N] _ [SPECIALITE] _ [ANNEE] .pdf

Les annexes (code source, captures d'ecran, certificats, rapports d'outils) doivent etre numerotees et referenciees dans le corps du texte. Elles ne sont pas comptabilisees dans le volume ci-dessus. La qualite de l'argumentation et la rigueur de l'analyse priment sur le volume.

Les deux competences transversales — maitrise de l'anglais technique et integration des principes du numerique responsable — doivent etre demonstrees dans l'ensemble du dossier et lors de la soutenance orale.

